

CS780 Capstone Project: OBELIX: the Warehouse Robot

Harsh Goel

251110034

harshgoel25@iitk.ac.in

Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This work focuses on training an OBELIX robot to find, attach, push, and unwedge a box in a partially observable environment. I primarily used a value-based approach built on D3QN-PER, extended with LSTM for temporal reasoning and ICM (forward + inverse models) for representation learning. Key innovations include reward shaping, intrinsic motivation, auxiliary heads (bad-state and bump detection), and noisy exploration. I experimented with PPO-style updates, knowledge distillation, and GAE, but the final model relied on stable Q-learning with improved representations. Results show significant task success (box pushing) despite suboptimal rewards, highlighting challenges in reward design and alignment in POMDP settings.

1 Competition Result

Codabench Username: h_251110034

Final leaderboard rank (Test Phase): 83

Leaderboard rank – Level 1 (static): 29

Leaderboard rank – Level 2 (blinking): 75

Leaderboard rank – Level 3 (moving + blinking): 24

Number of valid submissions made (Development(Level 1 , Level 2, Level 3) and Test (Final Submission Phase)):

Phase	Level / Description	Submissions
Development	Level 1 (Static Box)	9
	Level 2 (Blinking Box)	4
	Level 3 (Moving + Blinking Box)	6
Development Total	All Levels	19
Test (Final)	All Difficulty Levels	6
Total Submissions	Dev & Test Phase	25

2 Introduction and Problem Description

- The objective of this project is to train a house-cleaning robot so that it can find and push it efficiently out of the room (or arena). So, the motivation here is to make this robot adapt or learn this complex partially observable environment where there are lots of challenges like first making sure that robot explores without hitting the walls or boundaries, then once attached then pushing it to the boundary keeping the box in alignment, and to make higher reward, finding the best shortest trajectory to the box or predicting its approximate location and taking the actions so as to get attached and push it out of arena.

- The core task involves a sequence of objectives: **find** $\rightarrow$ **attach** $\rightarrow$ **push** $\rightarrow$ **unwedge** a grey box and move it to the boundary of the arena. **Finding** the box in the partially observable environment where the robot knows little information through sensor readings whether the box is in range of its sensors or whether there is any box/wall ahead or whether it gets stuck is difficult. Since based on this information, it has to move in the search space without hitting the walls/boundaries and when sensors detect it then it has to move/follow it and should **attach** to it. Once it gets attached then it has to **push** the box in proper alignment by keeping the box in the direction of movement and then finally **unwedge** it to the boundary. This requires both perception and long-term decision making.
- The environment provides an 18-dimensional binary observation space consisting of sonar sensors (near and far detection), along with bump and stuck bits. The action space is discrete with five actions corresponding to different movement directions.
- Three difficulty levels are considered:
 - **Level 1: Static Box** - the box does not move.
 - **Level 2: Blinking Box** - the box intermittently disappears, making interpretation more challenging for the robot in this POMDP where rewards are sparse.
 - **Level 3: Moving + Blinking Box** - the box moves dynamically while also blinking, making tracking significantly harder.

Additionally, wall obstacles further increase navigation complexity since there is only one bit to know whether there is a robot or a wall present ahead. So navigation / interpretation becomes more challenging and difficult there.

- This problem is inherently a **Partially Observable Markov Decision Process (POMDP)** because robot only sees within the range allowed by its sensors. If the box is far from it, it may not even know exactly where the box is. It would first need to explore a bit to get within the box's range. and doing this, requires past context and better representation learning which is difficult here.
- The **high-level objective** is to maximize the mean cumulative reward across all difficulty levels while ensuring robustness in the presence of walls and dynamic obstacles.

3 Approach and Model Evolution

- **Baseline / Initial Attempts:**
 - In first week of phase1 (where reward policy was different), I started with the random policy first and then tried **DDQN with MSE Loss and Huber Loss**. Not seen any learning signal due to insufficient exploration, so then tried **D3QN-PER with MSE Loss and Huber Loss**, and here I found Huber Loss to work slightly better over MSE.
 - Then I **kept D3QN-PER as my baseline** and tried improvements later in subsequent phases after trying out / exploring other standard algorithms to check which suits more to this environment.
- **Evolution Across Phases:**
 - **Phase 1 (Static Box):** (Reporting submissions from second week after reward policy fixed)
 - * Tried **D3QN-PER** (Huber Loss) first here and got the best rewards than other submissions tried in Phase 1.

- * Then I tried **Reinforce** without any baseline and it gave me worst rewards then went on trying **VPG** which improved the rewards because of introducing stable baseline and entropy loss for better exploration and stable learning.
 - * Then I tried **A3C** belonging to Actor-Critic family over two different configurations and observed almost similar results.
- **Phase 2 (Blinking Box):**
- * Tried **PPO** and it performed well in no-wall settings but struggled with walls because of ambiguity in determining wall/box ahead.
 - * **SAC** and **A2C** were tested but underperformed due to lack of proper representation learning and also sparse rewards were limiting proper searching and sticking to walls for some cases.
 - * **D3QN-PER** remained the most stable baseline but as I mentioned above problem was with better representations since 18 bits state vector was not alone sufficient in well-planned searching and following the target.
- **Phase 3 (Moving + Blinking Box):**
- * Used **LSTM in D3QN-PER’s Duel Network** to handle partial observability by keeping the past context and added priority based Sequence Replay Buffer for efficient sampling to improve exploration.
 - * Above algorithm made the robot to follow the box although still representations were not efficient since even after getting stuck to box, it remained idle. So I **added action mask head** determining which action to take in different scenarios and tried **decay epsilon exploration** switching between **sampling action from action mask’s distribution and greedy action selection**. But problem was inefficient mapping of actions to partially observable latent representations.
 - * Incorporated **representation learning via forward and inverse models (ICM)** [1] and along with this, I did **reward shaping and added intrinsic reward** for changing the states in order to prevent spinning. And **later** I need to **decay intrinsic reward’s coefficient over episodes** to prevent exploitation of trivial behaviors (e.g., spinning or wall hitting).
 - * Introduced auxiliary heads:
 - **Bad state detection** (to avoid walls/stuck states)
 - **Bump classification head** (distinguishing wall vs box interaction)

This change helped the robot to find, push and unwedge box to the boundary but still **suffered from hitting the wall** and eventually leading to poor rewards but **in terms of success, it performed well**.
- **Test Phase:**
- * To prevent searching being in contact with wall, I introduced **stuck penalty** (when infrared bit is set and sensors didn’t detected the box) and also introduced **alignment bonus** to keep the box ahead of the robot while pushing. But eventually, combination of these two made it to spin at one place.
 - * Then I tried using **TD(λ) instead of TD(0) for better credit assignment** in above algorithm, but eventually leads to bad reward because of non-distinguishing wall/box problem (not pushing box even gets attached).
 - * Then I tried **distillation learning** making 4 of my previous models the teacher (where one is exploring without hitting wall, another is pushing the box to boundary efficiently, another has learnt some good trajectories like passing smoothly between walls without touching and last one is trying out other options if alignment creates trouble instead of

keep hitting the wall), then I used **LSTM [ICM] network with uniform Sequence Replay Buffer** for my model to train by keeping its Q values' distribution as close as to the parent's Q values distribution and also used **PPO style updates** inorder to prevent Q values from changing too rapidly (making it robust to outliers).

- * Introduced **GAE(λ) for advantage estimation** for better credit assignment and improving exploration, and it helped in making better rewards than before since last time for PPO Loss instead of using advantages, I used TD error.
- * Then I further went back and modified phase 3 last submission algorithm (since that one succeeds 80% of the times) by changing the logic for deciding stuck loss, bad_head.loss and bump.loss.

Detailed description of final / best model(s)

- **Algorithm family:** Final model is based on **D3QN-PER**, extended with **LSTM** for partial observability and **ICM (forward + inverse models)** for representation learning. Compared to PPO/A2C/SAC, value-based methods were more stable and sample-efficient in this discrete setting.
- **Network architecture:**

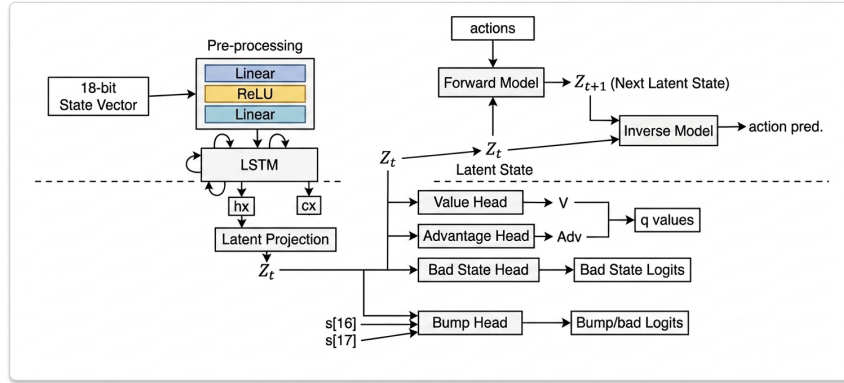


Figure 1: Best Model Network Architecture

- **Exploration strategy:** Used ϵ -greedy with **biased random exploration** and **noisy Q-values**. During exploration (ϵ case), actions are sampled from a fixed distribution giving higher probability to forward movement. During exploitation, added **Gaussian noise to Q-values** with **decaying standard deviation** over episodes, allowing more exploration early and stable decisions later. This helped reduce random spinning and encouraged meaningful movement (especially forward).
- **Reward shaping:** Used **log-scaled rewards**, **intrinsic reward** (forward model error), **progress reward** (sensor-based), and **penalties** for bad states. Intrinsic reward was decayed over time to avoid degenerate behaviors like wall hitting.
- **Hierarchical / modular ideas:** Instead of explicit hierarchy, used **modular learning via auxiliary heads**: representation (ICM), safety (bad-state + bump), and task (Q-learning). This improved both exploration and stability.

4 Experiments and Implementation Details

Training Setup:

Phase	Episodes / Timesteps	Max Steps	Hardware	Time
Phase 1	150–300 episodes	500–1000	GPU	~2–4 hrs/model
Phase 2	5e6–1e7 timesteps & 500 episodes	1000	GPU	~6–10 hrs/model
Phase 3	200–800 episodes	1000	GPU	~8–12 hrs/model
Test Phase	500–1000 episodes	1000	GPU	~8–12 hrs/model

Core Hyperparameters Across Models:

Parameter	Range Used
Discount factor (γ)	0.99 – 0.999
Learning rate	$1e^{-4} - 1e^{-2}$
Batch size	32 – 64
Replay buffer	$10^5 - 10^6$
Target update (τ)	0.001 – 0.35
Exploration (ϵ)	$1.0 \rightarrow 0.001$ (decay)
Sequence length (LSTM)	20 – 50
Burn-in	20 – 40

Major Model Variants (Condensed Config Table):

Model	LR	γ	Batch	Buffer	Key Features
DDQN	2.2e-3	0.999	64	1e5	baseline
D3QN-PER	1e-2	0.999	64	1e5	PER + dueling
REINFORCE	1e-3	0.999	–	–	no baseline
VPG	1e-3	0.999	–	–	value + policy net
A3C	1e-4	0.995–0.999	–	–	multi-worker
PPO	3e-4	0.995	–	–	clipping + GAE
SAC	3e-4	0.99	64	1e5	entropy regularization
A2C	1e-4	0.99	–	–	synchronous AC
D3QN-PER+LSTM	7e-4	0.999	64	1e5	seq learning
+ Action Mask	3e-4	0.999	32	1e5	learned masking
- Action Mask + ICM	7e-4	0.99	64	1e6	intrinsic reward
+ Aux Heads + Noisy Exp.	5e-4	0.99	64	1e6	bad/bump heads
+ GAE	5e-4	0.99	64	1e6	better credit assignment
Knowledge Distillation + PPO	1e-4	0.99	32	1e6	clipping + GAE + prior

Exploration Strategy:

- ϵ -greedy (linear + exponential decay)
- Noisy exploration with defined probabilities with FW having higher probability (final phase)
- Entropy regularization (PPO, A2C, SAC)

Multi-task / Curriculum Learning:

- No explicit joint training across difficulties. But focus majorly on training my robot with difficulty 3 because if it adapts itself to that environment then it will also performed well on lower difficulties.
- Instead I trained my agent with randomly switching between wall and no-wall settings for better generalization on difficulty 3 from phase 3 onwards.

Local Evaluation Strategy:

- Evaluated using 5 episodes per submission with unique seeds.
- Tested on both wall and no-wall settings.
- Fixed seeds for reproducibility + varied configs for robustness
- Avoided overfitting by not tuning exclusively on a single environment settings.

5 Results

CR: Cummulative Reward

- **Development Phase Performance:**

Model	CR With Walls	CR Without Walls
Phase 1		
D3QN-PER	-1980.7	-1989.2
VPG	-81428.9	-1990.9
A3C	-1980.7	-1989.2
Phase 2		
PPO	-350703.8	-645.8
D3QN-PER	-1985.6	-1990.9
SAC	-7939.1	-3226.9
A2C	-318937.9	-1573.4
Phase 3		
D3QN-PER+LSTM	-49209.1	-1491.8
+ ICM	-1530.1	-1491.3
+ Aux.Heads [ICM_AUX]	-1529.4	-1491.8

- **Final Test Phase Results (All Levels Combined):**

Model	Walls (P1/P2/P3)	No Walls (P1/P2/P3)	Weighted
Test Phase			
[ICM_AUX]+Align.Bonus	-1980.7 / -1980.7 / -1531.3	-1990.1 / -1989.2 / -1491.3	-1827.95
+Fixed Stuck-Align	-1979.3 / -1978.8 / -2722.3	-1986.3 / -22255.9 / -27224.9	-8198.36
KD+PPO	-23458.4 / -1976.7 / -1539.3	-25282.7 / -4464.1 / -1480.7	-9558.55
KD+PPO+GAE	-1981.2 / -1981.2 / -1530.9	-1990.9 / -1990.9 / -1491.3	-1828.41

- **Learning Curves:**

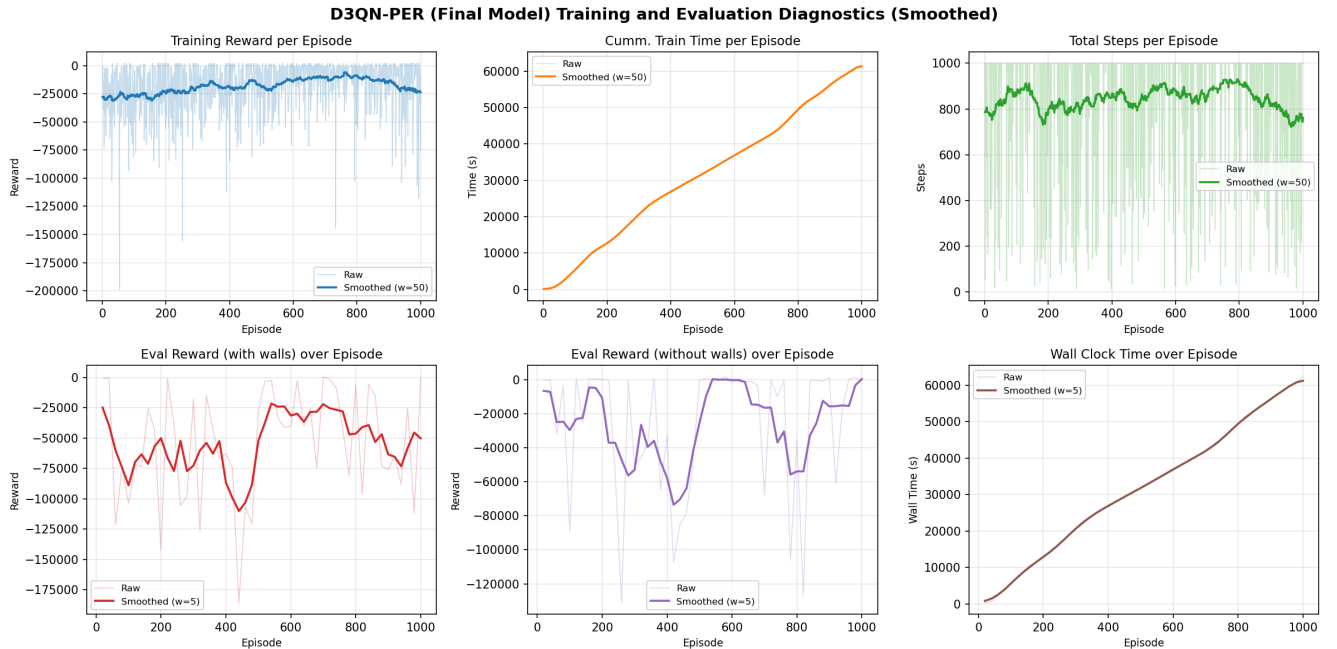


Figure 2: Training and Evaluation Diagnostics

- **Best Performing Model:**

- I find **ICM AUX with Stuck Penalty and Alignment Bonus** to be **better in terms of accomplishing the goal** but it struggles **with no walls** because it finds another way of receiving alignment bonus decided by me. It pushes the box but if alignment is not proper then after being attached to boundary, it kept the box in front and eventually pushes it to orthogonal boundary. So i considered it as my best model (that is highlighted above) on basis of successful attempts better than the first submission in eval phase giving better reward.
- Earlier, I was giving a continuous reward based on change in infrared bit but that eventually made it to spin, in order to make higher rewards. Later to fix it, give alignment reward when sensors detected the box and infrared bit was set. But later found, it again finds some easiest option to gain this reward. Like being stuck with wall and attached to box from either left or right and pushing it.
- But still these models outperform other in terms of better **representation learning, sample efficiency (PER) and robustness (dual training env)**. But suffers from **box-wall alignment** problem and also **lacks proper exploration** because of sparse reward structure.

Github link for all the models tried: [Code Here](#)

Video demonstration for best performing model: [Video Demo](#)

6 Error Analysis and Discussion

Common Failure Modes:

- **Level 1 (Static Box):** Even though the box is static, failures occur due to **poor exploration early in episodes** and **incorrect alignment during pushing**. The agent sometimes reaches the box but fails to maintain direction, leading to inefficient pushing or getting stuck near boundaries. Eventually in order to avoid the penalty, it only spins.
- **Level 2 (Blinking Box):** The main issue is **loss of track of the box**. Since observations are partial, the agent sometimes treats blinking as disappearance and starts re-exploring unnecessarily. This leads to **false negatives** and unstable behavior, especially when the box reappears in a different relative position. But again **problem was with the alignment**, because of which agent ultimately spins only and stops looking out for the box.
- **Level 3 (Moving + Blinking Box):** Here the problem becomes harder due to **prediction and interception**. The agent often fails to estimate the motion of the box correctly and ends up following outdated trajectories. Even with LSTM and ICM, **long-term prediction remains weak**, leading to delayed or misaligned pushing.
- **With Walls:** Navigation becomes ambiguous due to **limited sensor information** or infrared bit (used for detecting wall/box both). The agent cannot clearly distinguish between wall and box in some cases, leading to:
 - Frequent **collisions and stuck states**
 - Exploiting reward shaping (e.g., alignment reward near walls)
 - Getting trapped in local loops or oscillating near the walls.

Limitations and Improvements:

- Reward shaping introduced **reward hacking**, where the agent finds shortcuts instead of solving the actual task. For ex., I tried fixing the alignment issue and give reward when infrared bit is set and front sensors also detects box and action taken is forward but it took this reward being in contact with the wall.
- Representation learning via ICM improved exploration but was still **not sufficient for precise control** because of the unawareness of its coordinates in the plane which can help more in learning meaningful representations.
- With more time, I would:
 - Introduce **model-based RL** for predicting box dynamics and exploring in order to minimize the euclidean distance between the two.
 - Design **more robust reward functions** favouring better exploration.
 - try out **hierarchical based RL** for conditioned exploration to maximize reward.

Surprising Findings:

- **Better rewards do not always mean better task success.** Some models with worse rewards actually exploring, pushing and terminates but at the cost of hitting or getting stuck to wall/boundary.
- **Value-based methods (D3QN-PER)** were significantly more stable than policy-gradient methods in this discrete setup.
- **Intrinsic reward needs careful decay**, otherwise the agent exploits it (e.g., spinning or hitting walls).
- Bad-state auxiliary head helped in learning **safety-aware behavior** by stopping itself when comes closer to wall or boundary.

7 Conclusion

- **Key Achievements:** I successfully developed a model that can **reliably find, attach, and push the box** in a challenging partially observable environment but only when box is in sensors' range. The final D3QN-PER + LSTM + ICM + auxiliary heads model showed strong **task-level success** (60–70% pushing), despite suboptimal reward scores. The work highlights the importance of **representation learning, exploration design, and reward shaping**.
- **Main Takeaway:** In partially observable robotic tasks, **memory and representation are more important than just algorithm choice**. Even strong RL algorithms like PPO, fail without proper state understanding. Also, **reward design is critical**—a poorly designed reward can completely mislead the agent.
- **Future Directions:**
 - RLHF can be used for giving feedback signals to train agent efficiently.
 - Also work can be done to focus on improving representations for better learning.

LLM Usage Declaration

- Which Large Language Model (LLM) did you use to complete the competition or write the report?
I took help of **ChatGPT** to complete this report as well as completing the competition.
- In which part did you use the LLM, and how did it help you?

Part	Used LLM? (Yes/No)	Purpose of Using LLM
Understanding concepts	Yes	Used to understand how replay buffer should change on using LSTM. Since with simpler architectures, replay buffer stores set of experiences but now with LSTM, it stores set of sequences of experiences so how priority based thing can be done.
Ideation / brainstorming	Yes	I took help of it for confirming the possibilities of the algorithm, I decided to implement.
Debugging small code snippets	Yes	Most of the times, I struggle with device mismatch thing since running my algorithm on GPU. Took help of it there.
Plotting / visualization code	Yes	For plotting the graphs efficiently, took its help.
Grammar / writing style	No	–
Report structure / section ideas	Yes	Took help of it for defining the proper report structure and also used it for creating the tables after providing him the configs and scores for each model.
Other (specify)	No	–

- Did you refer to any other sources or websites apart from the LLM and lecture slides?
If yes, please mention clearly (papers, blogs, GitHub repos, textbooks etc.)

Yes I have referred to following references mentioned below:

- Curiosity-driven Exploration by Self-supervised Prediction [1]
- Used PyTorch [2] and Matplotlib [3] Documentation.
- CS780 Lecture Notes

- Apart from any LLM usage, what was your own contribution?

Briefly describe your personal understanding, reasoning, implementation effort, analysis, and experimentation.

My contribution includes:

- Implemented all algorithms by myself, and observed why won't they work as expected.
- LLM was only for guiding me a bit whether I was thinking in right direction that could make the robot explores irrespective of sparse reward policy.
- Also it only helped me in finding the relevant papers over the net. Later I read, understood and used them if they were really useful.
- Whole report apart from plots, architecture diagram and tables is written by me only.

References

- [1] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, “Curiosity-driven exploration by self-supervised prediction,” in *Proceedings of the IEEE Conference on Computer Vision Workshops*, 2017.
- [2] A. Paszke *et al.*, “Pytorch: An imperative style, high-performance deep learning library.” <https://pytorch.org/>, 2019.
- [3] J. D. Hunter, “Matplotlib: Visualization with python.” <https://matplotlib.org/>, 2007.